

# Deep Learning and Natural Language Processing

## Lecture 13: The Transformer

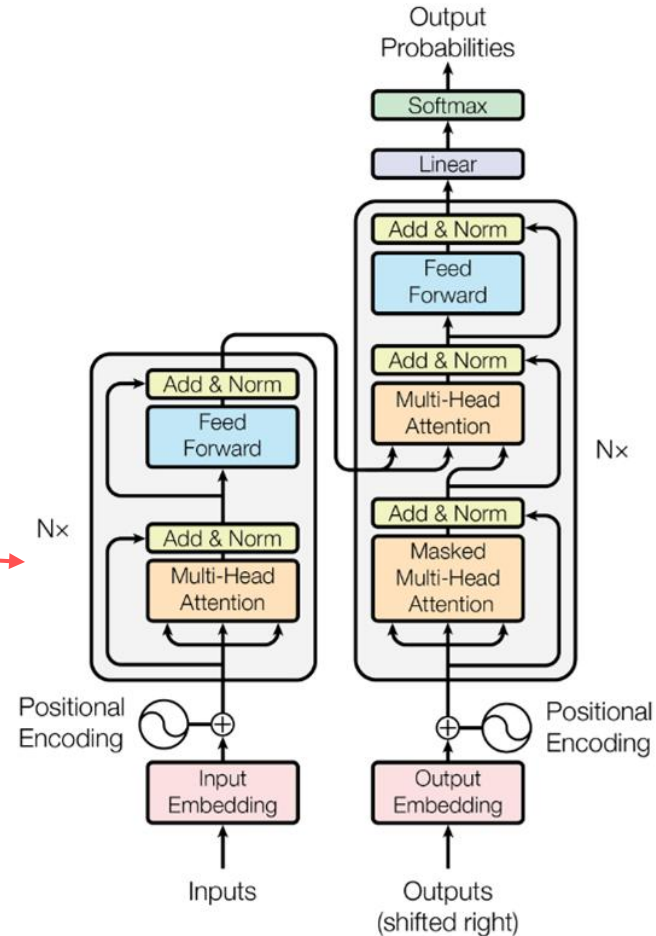
Prof. Dr. Alan Akbik  
Lehrstuhl "Maschinelles Lernen"

# Last Lecture

Introduced the **transformer**!

Goal: Understand this image

- [\[Vaswani et al., 2017\]](#)



# Transformers for Translation



Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

| Model                           | BLEU        |              | Training Cost (FLOPs)                 |                     |
|---------------------------------|-------------|--------------|---------------------------------------|---------------------|
|                                 | EN-DE       | EN-FR        | EN-DE                                 | EN-FR               |
| ByteNet [15]                    | 23.75       |              |                                       |                     |
| Deep-Att + PosUnk [32]          |             | 39.2         |                                       | $1.0 \cdot 10^{20}$ |
| GNMT + RL [31]                  | 24.6        | 39.92        | $2.3 \cdot 10^{19}$                   | $1.4 \cdot 10^{20}$ |
| ConvS2S [8]                     | 25.16       | 40.46        | $9.6 \cdot 10^{18}$                   | $1.5 \cdot 10^{20}$ |
| MoE [26]                        | 26.03       | 40.56        | $2.0 \cdot 10^{19}$                   | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [32] |             | 40.4         |                                       | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [31]         | 26.30       | 41.16        | $1.8 \cdot 10^{20}$                   | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [8]            | 26.36       | <b>41.29</b> | $7.7 \cdot 10^{19}$                   | $1.2 \cdot 10^{21}$ |
| Transformer (base model)        | 27.3        | 38.1         | <b><math>3.3 \cdot 10^{18}</math></b> |                     |
| Transformer (big)               | <b>28.4</b> | <b>41.0</b>  | $2.3 \cdot 10^{19}$                   |                     |

Significantly outperformed prior work!

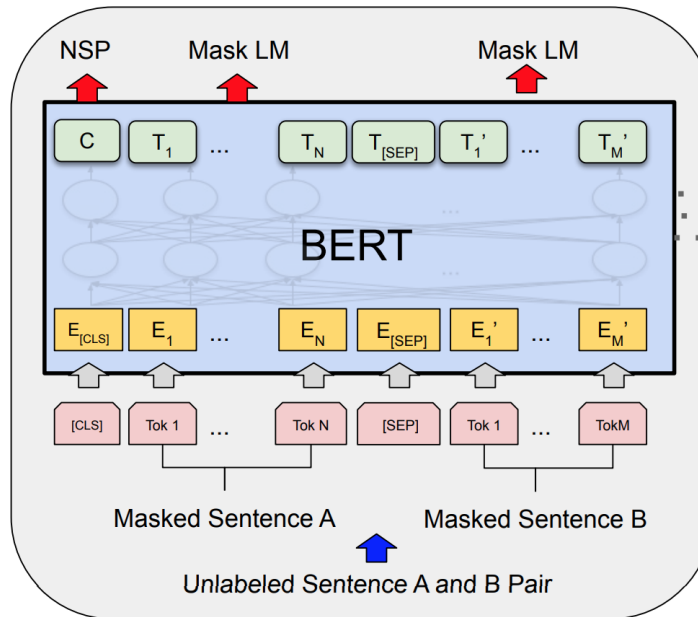
What about other tasks?

# This Lecture

## Transfer learning from language models

### Topics:

- Textual Entailment
- Transformer language models
- GPT
- BERT
- Fine-Tuning vs Linear Probing



# Outline

Recap: Prior Knowledge



# Recap: Prior Knowledge

Idea: Rather than start from scratch, initialize network with “prior knowledge”

Models of **word-level knowledge**

- e.g. knowledge that could be **useful for any task**

Pre-trained word embeddings



Factual beliefs about the world held by language model. Generated by Midjourney (2023).

# Word Embeddings



## Similarities of words

```
# load wikipedia embeddings
wiki_vectors = api.load('glove-wiki-gigaword-100')

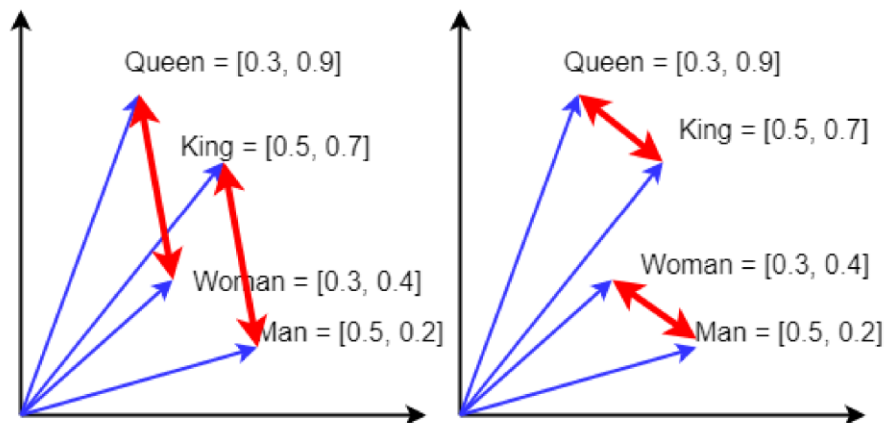
wiki_vectors.most_similar("castle")

[('manor', 0.7622005939483643),
 ('mansion', 0.6870322823524475),
 ('fortress', 0.6852314472198486),
 ('castles', 0.6557385921478271),
 ('palace', 0.6393353939056396),
 ('abbey', 0.6129941940307617),
 ('chapel', 0.6068748831748962),
 ('windsor', 0.5988348126411438),
 ('medieval', 0.5938449501991272),
 ('built', 0.582231342792511)]
```

# Word Embeddings

Visualizations indicate that embeddings encode various semantic and syntactic information

**Analogies:** “king” - “man” + “woman” = “queen”



[Image source](#)

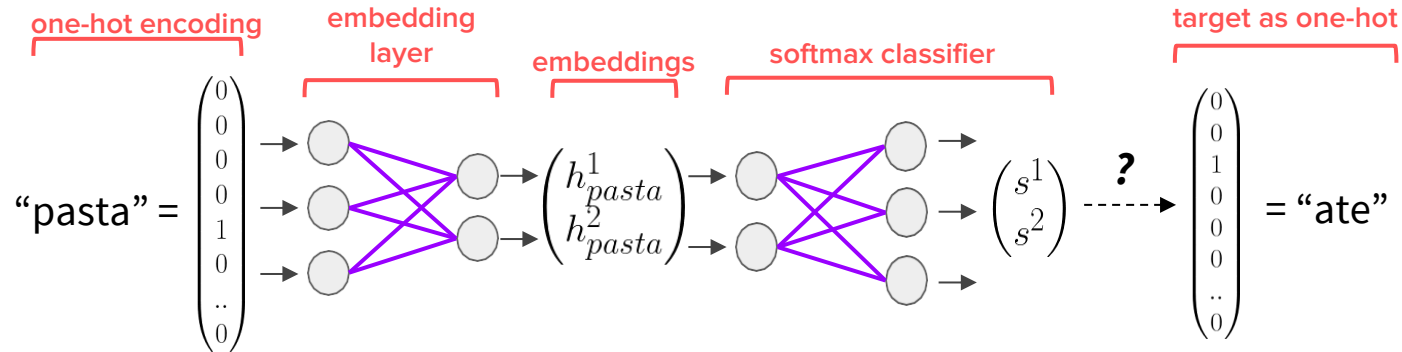


# Word2Vec



Word2Vec can be trained over plain text data – no manual annotations needed!

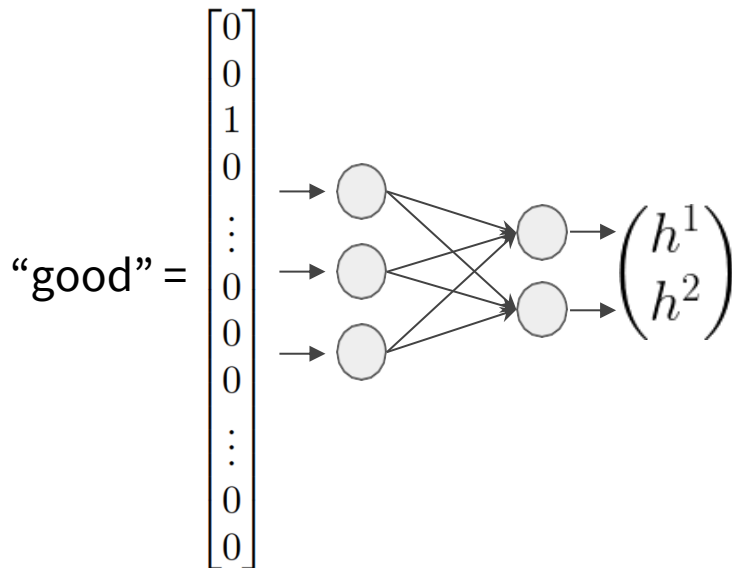
Can be trained for huge vocabulary



# Model Initialization

Model training (see lecture 2): All learnable parameters are randomly initialized

Embedding layer: starts random, learns how to embed words for PoS tagging

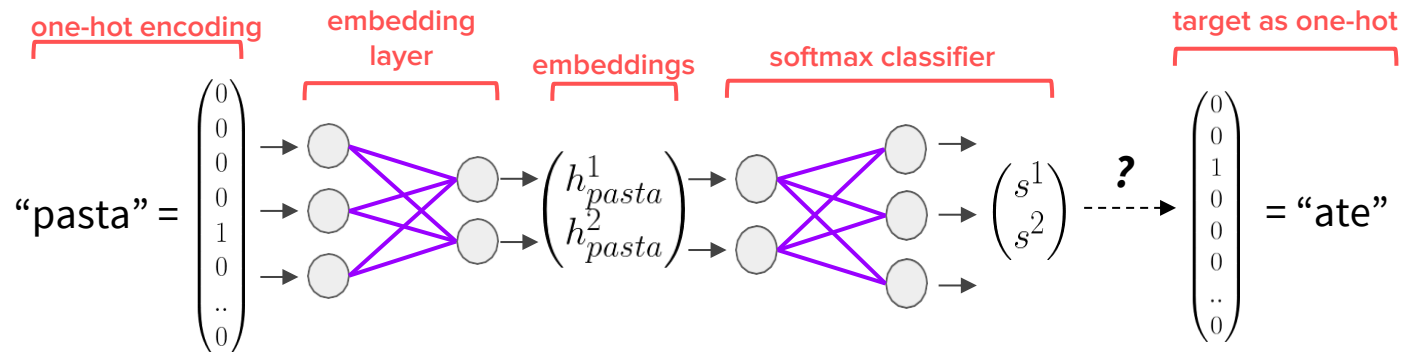


Problem: for most words,  
not enough training data

# Idea: Use Word2Vec Embeddings as Init

Word2Vec can be trained over plain text data – no manual annotations needed!

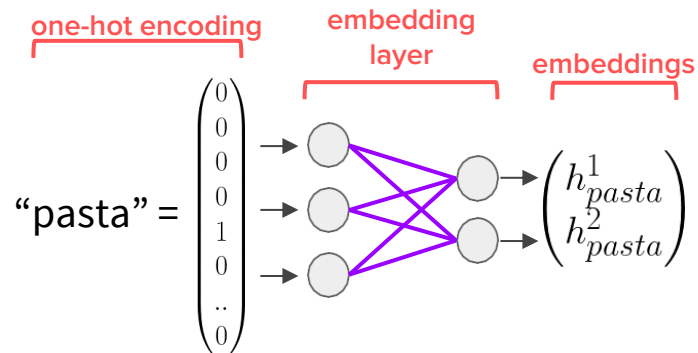
Can be trained for huge vocabulary



# Idea: Use Word2Vec Embeddings as Init

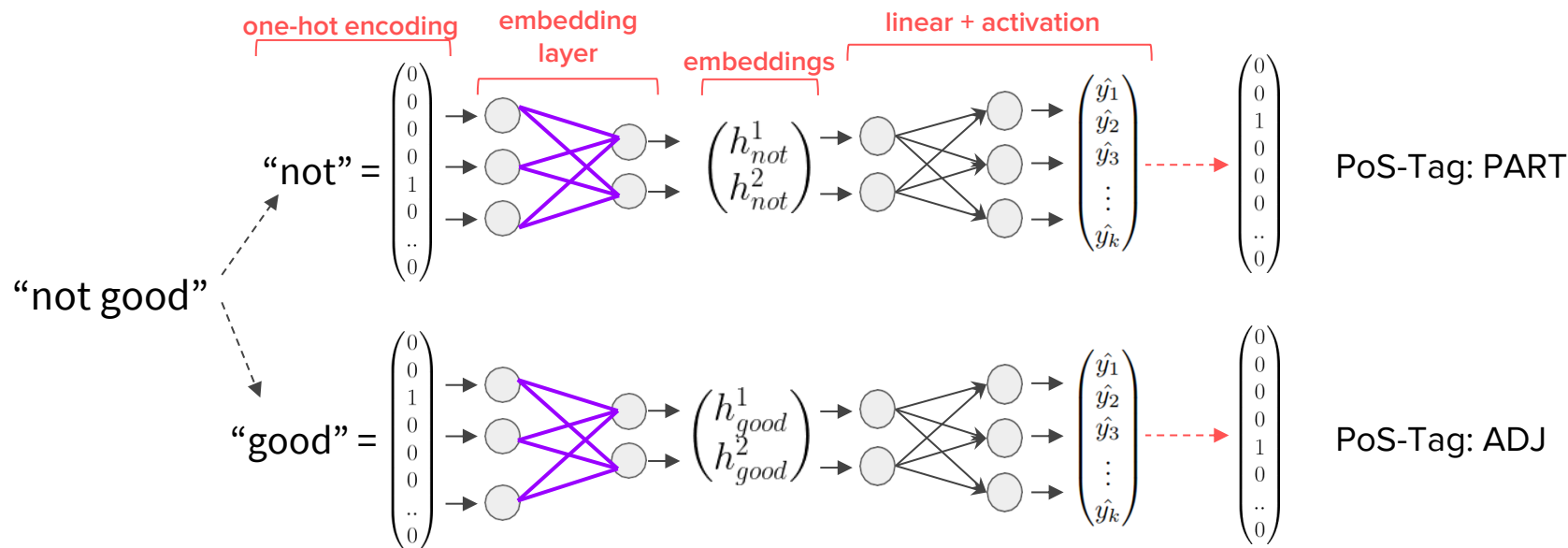
Word2Vec can be trained over plain text data – no manual annotations needed!

Can be trained for huge vocabulary



# Naive PoS-Tagger

Naive sequence labeling, but initialize embedding layer with word2vec weights!



# Naive PoS-Tagger with Embeddings

The true power of word embeddings: initializing the embedding layer of other networks

Until transformer language models (2019), all state-of-the-art approaches in NLP used word embeddings!

Example in homework:

| Approach            | Accuracy |
|---------------------|----------|
| Naive               | 0.76     |
| Naive w. Embeddings | 0.84     |

# Limitations?



Word embeddings infuse our model with prior knowledge on words

Limitations?

# Outline

## Natural Language Inference

### Transfer Learning

- Causal Language Modeling (GPT)
- Masked Language Modeling (BERT)

### Fine-Tuning vs Linear Probing





# GLUE Benchmark



GLUE Benchmark [\[Wang et al., 2018\]](#)

- GLUE = General Language Understanding Evaluation (<https://gluebenchmark.com/>)
- 11 different NLP tasks, mostly NLI (natural language inference)
- two we already know: CoLA and Winograd Schema Challenge (rephrased as NLI)

Unlike earlier challenges, they never released the gold labels for the test data

- you train a model on train/dev, and upload your predictions to their server
- then you get a score
- **gives researchers less opportunity to overfit to specifics of the test data**
- systems are ranked on their [leaderboard](#)

# GLUE Benchmark: Leaderboard



| Rank Name                                                                         |                                     | Model                 | URL                                                                                 | Score | CoLA | SST-2 | MRPC      | STS-B     |
|-----------------------------------------------------------------------------------|-------------------------------------|-----------------------|-------------------------------------------------------------------------------------|-------|------|-------|-----------|-----------|
| 1                                                                                 | JDExplore d-team                    | Vega v1               |                                                                                     | 91.3  | 73.8 | 97.9  | 94.5/92.6 | 93.5/93.1 |
| 2                                                                                 | Microsoft Alexander v-team          | Turing NLR v5         |                                                                                     | 91.2  | 72.6 | 97.6  | 93.8/91.7 | 93.7/93.3 |
| 3                                                                                 | ERNIE Team - Baidu                  | ERNIE                 |  | 91.1  | 75.5 | 97.8  | 93.9/91.8 | 93.0/92.6 |
| 4                                                                                 | DIRL Team                           | DeBERTa + CLEVER      |                                                                                     | 91.0  | 74.5 | 97.5  | 93.3/91.0 | 93.4/93.1 |
| 5                                                                                 | AliceMind & DIRL                    | StructBERT + CLEVER   |  | 91.0  | 75.3 | 97.7  | 93.9/91.9 | 93.5/93.1 |
| 6                                                                                 | DeBERTa Team - Microsoft            | DeBERTa / TuringNLRv4 |  | 90.8  | 71.5 | 97.5  | 94.0/92.0 | 92.9/92.6 |
| 7                                                                                 | HFL iFLYTEK                         | MacALBERT + DKM       |                                                                                     | 90.7  | 74.8 | 97.0  | 94.5/92.6 | 92.8/92.6 |
|  | 8                                   | PING-AN Omni-Sinitic  |                                                                                     | 90.6  | 73.5 | 97.2  | 94.0/92.0 | 93.0/92.4 |
| 9                                                                                 | T5 Team - Google                    | T5                    |  | 90.3  | 71.6 | 97.5  | 92.8/90.4 | 93.1/92.8 |
| 10                                                                                | Microsoft D365 AI & MSR AI & GATECH | MT-DNN-SMART          |  | 89.9  | 69.5 | 97.5  | 93.7/91.6 | 92.9/92.5 |

# Winograd Schema Challenge



a) I poured water from the bottle into the cup until it was *full*.

bottle or cup?

b) I poured water from the bottle into the cup until it was *empty*.

bottle or cup?

# Winograd Schema as NLI

Natural Language Inference: label pairs of sentences as entailing/contradictory/neutral

|             |                             |                       |
|-------------|-----------------------------|-----------------------|
| Premise:    | I poured water into the cup | ] -> <b>entailing</b> |
| Hypothesis: | The cup became full         |                       |

|             |                             |                           |
|-------------|-----------------------------|---------------------------|
| Premise:    | I poured water into the cup | ] -> <b>contradictory</b> |
| Hypothesis: | The cup became empty        |                           |

This is a modified version of the Winograd Schema Challenge, rephrased as NLI

# Natural Language Inference



Natural Language Inference: label pairs of sentences as entailing/contradictory/neutral

|             |                                                |                           |
|-------------|------------------------------------------------|---------------------------|
| Premise:    | The trophy doesn't fit into the brown suitcase | ] -> <b>entailing</b>     |
| Hypothesis: | The trophy is too large                        |                           |
| Premise:    | The trophy doesn't fit into the brown suitcase | ] -> <b>contradictory</b> |
| Hypothesis: | The suitcase is too large                      |                           |
| Premise:    | The trophy doesn't fit into the brown suitcase | ] -> <b>neutral</b>       |
| Hypothesis: | The suitcase is borrowed                       |                           |

# Natural Language Inference

## NLI as classification

- Input: 2 sentences
- Output: one of 3 classes  
(entailing / contradictory / neutral)

## Similar to sentiment classification

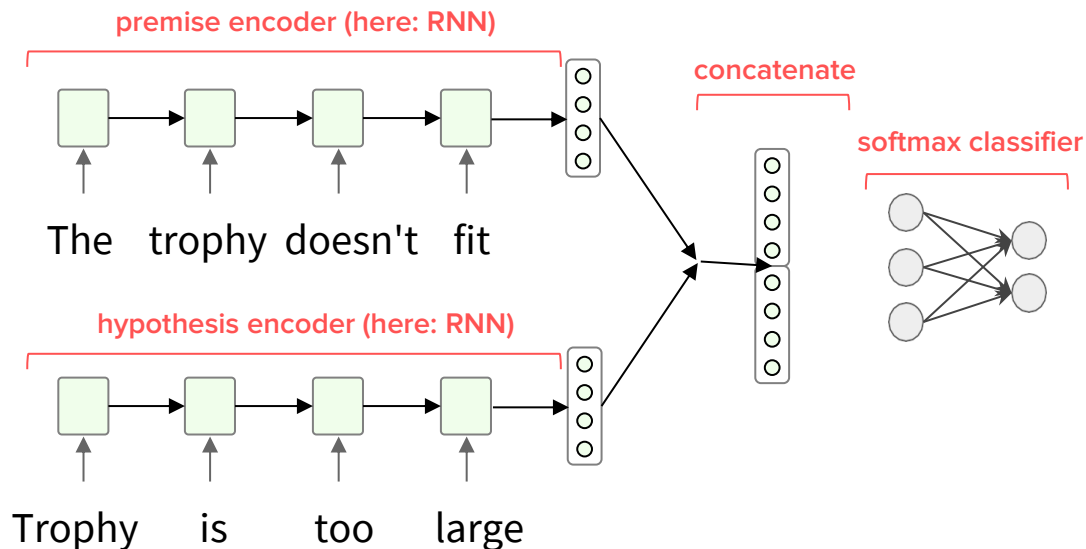
- input: 1 sentence
- output: one of 3 classes  
(positive / negative / neutral)

**BUT: In NLI we have two inputs. How does the neural architecture look like then?**

# Option 1: Dual Encoder Architecture

Two encoders: one for premise, one for hypothesis

- concat their outputs for softmax classifier



**But that's so 2018!**

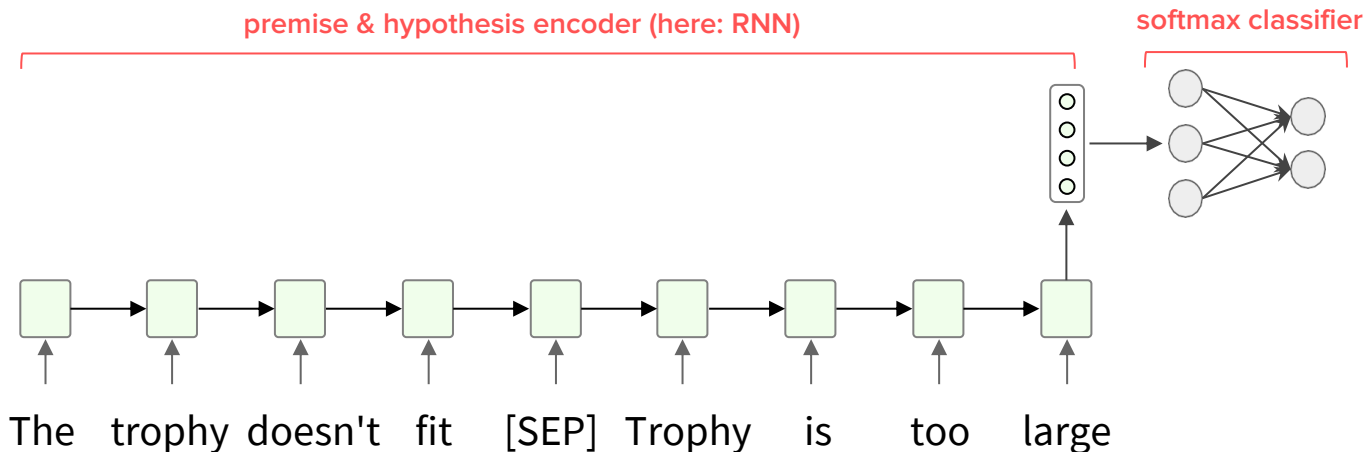
Limitation: Encodes premise and hypothesis separately :(

Only separately encode when you have to

# Option 2: Encode as a Single String

Idea: Make a single string out of premise and hypothesis

- place a special “separator token” between both
- use one encoder for both!
- even better: use transformer (attention between all words)

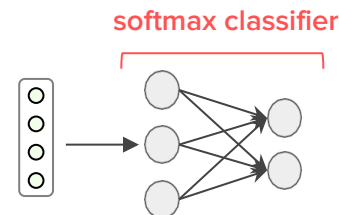
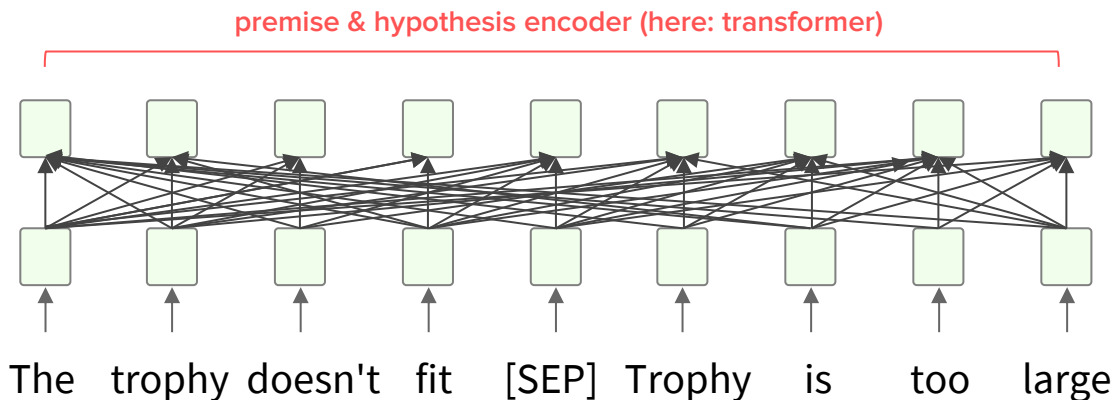




# Option 2: Encode as a Single String

Idea: Make a single string out of premise and hypothesis

- place a special “separator token” between both
- use one encoder for both!
- even better: use transformer (attention between all words)

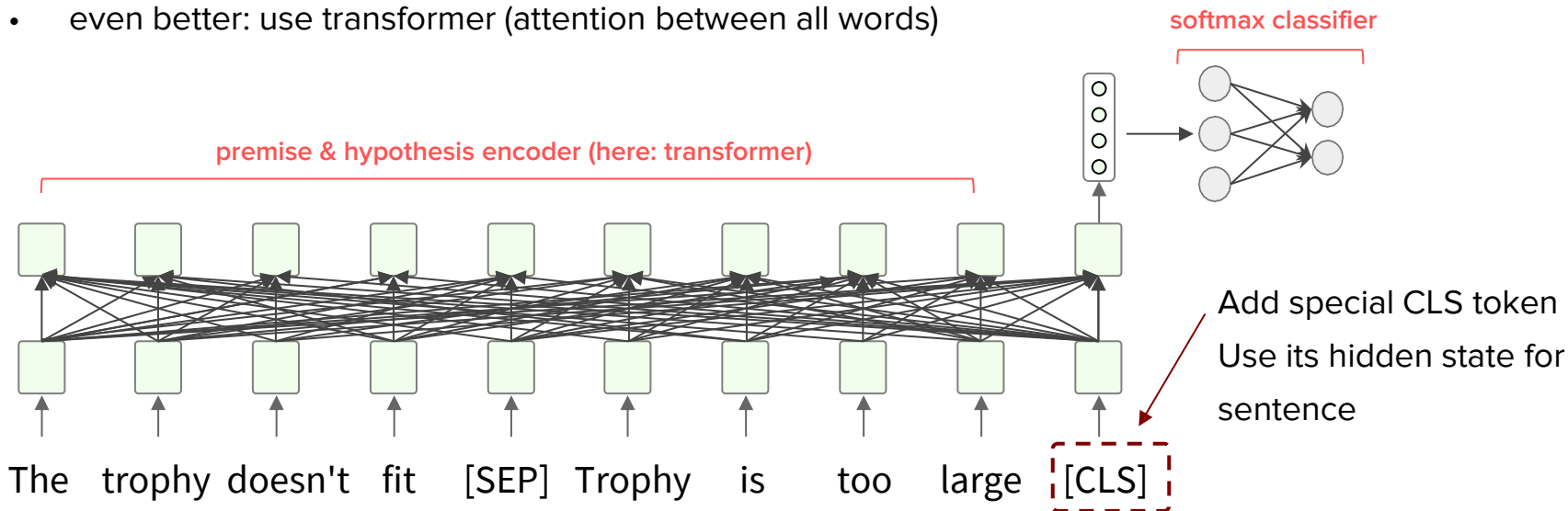


Wait! What state to use to represent sentence?

# Option 2: Encode as a Single String

Idea: Make a single string out of premise and hypothesis

- place a special “separator token” between both
- use one encoder for both!
- even better: use transformer (attention between all words)



# Difficulty of Natural Language Inference

With sentiment analysis, a “trivial” solution might get us quite far

- learn positive/negative words and bigrams
- with enough training data, could be learned from scratch
- **in our exercises: even bag-of-words did pretty well!**

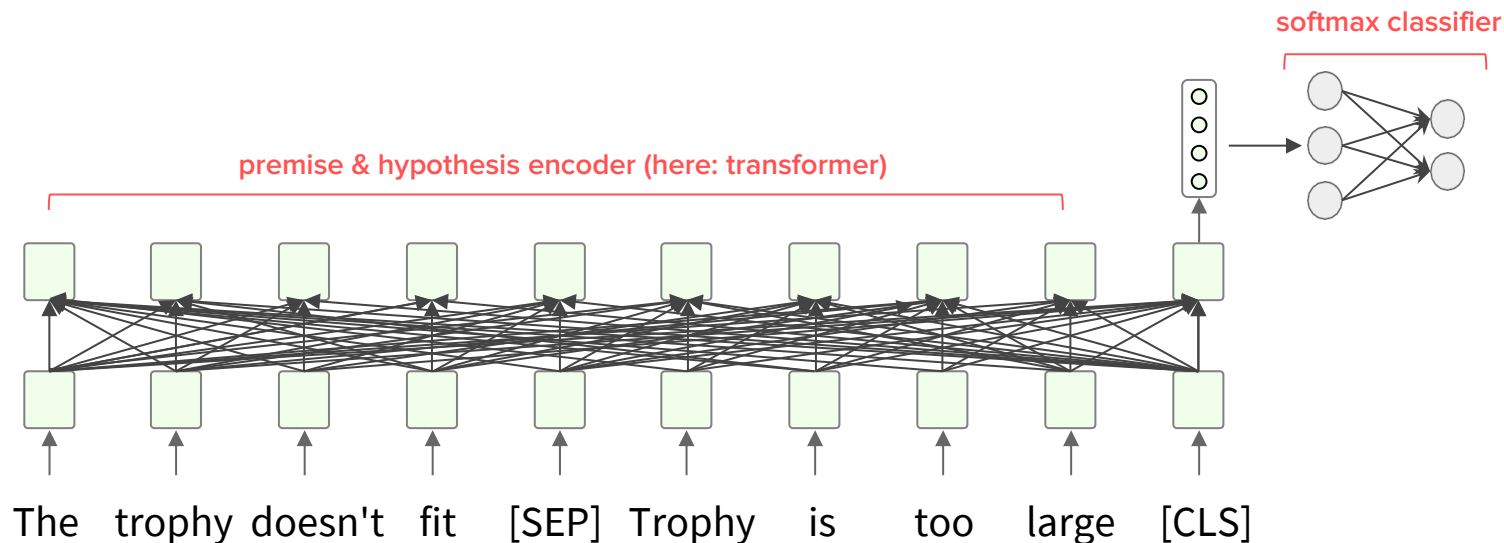
Natural language inference

- any two statements (could be about anything)
- relative: decide whether statements logically follow one another
- how would a trivial solution look like here? Trivial solution not possible?
- **how much training data do we need to solve this?**

**Learning a general entailment model from scratch infeasible**

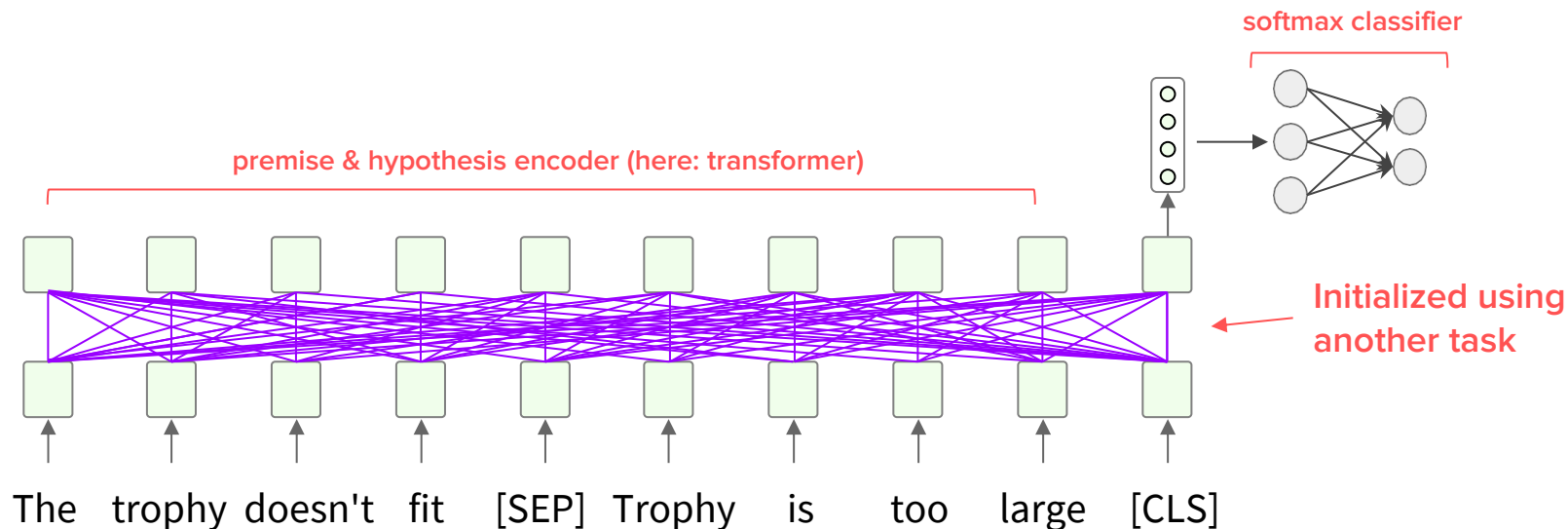
# Missing Ingredient: World Knowledge

Transfer learning: Can we initialize the weights of the RNN / transformer network with another task? Like word embeddings but for entire sentences?



# Missing Ingredient: World Knowledge

Transfer learning: Can we initialize the weights of the RNN / transformer network with another task? Like word embeddings but for entire sentences?



# Outline

New Task: Natural Language Inference

## Transfer Learning

- Causal Language Modeling (GPT)
- Masked Language Modeling (BERT)

Fine-Tuning vs Linear Probing



# Transfer Learning

## Classic machine learning

- randomly initializes all network weights
- “empty” model that is trained from scratch using training data
- can only learn what is in the training data

## But is this how humans learn?

- *not really*: humans already know many things when learning a new task

## Transfer learning

- instead of training from scratch, use an **already trained network**
- **pre-training**: network trained for a different task, so weights make sense for this task
- use training data for new task to **slightly modify** weights of pre-trained model
- **modification instead of training from scratch**

# Pre-Training



What is a good pre-training task for NLP?

Desiderata

- usable to train a large transformer or RNN architecture
- it should be **exceedingly difficult**: require general language understanding to solve
- there should be **near-endless amounts of training data** available

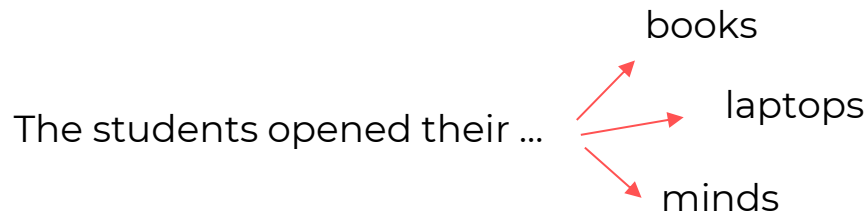




# Language Modeling



Language modeling is the task of **predicting what word comes next**



More formally: given a sequence of words  $x^1, x^2, \dots, x^t$

compute the probability distribution of the **next word**  $x^{t+1} : P(x^{t+1} | x^t, \dots, x^1)$

where  $x^{t+1}$  can be any word in the vocabulary  $V = \{w_1, \dots, w_{|V|}\}$

# Language Modeling for Transfer Learning



Language modeling as a **sentence-level pre-training objective**

- Endless amounts of training data
- **Very hard: You can only predict the next word well if you “understand” the sequence that comes before it!**

## **Examples:**

- I poured water into the cup until it was \_\_\_\_
- The trophy didn't fit because it was too \_\_\_\_
- In the zoo, I saw a \_\_\_\_

# Training Data

Training data for language modeling requires no extra labels (like PoS or sentiment tags)

- Rather, the words themselves are the labels
- **Raw text data is enough to train!**

Example raw text: I poured water into the cup.

- \_\_\_\_
- I \_\_\_\_
- I poured \_\_\_\_
- I poured water \_\_\_\_
- I poured water into \_\_\_\_
- I poured water into the \_\_\_\_

# LSTM Language Models

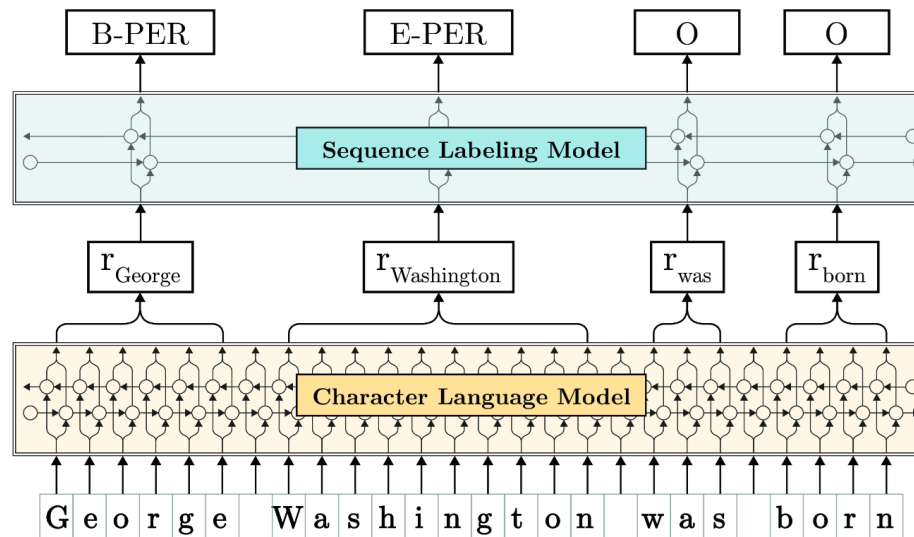
“Early” work used LSTM language models for transfer learning (ELMo and Flair)

Flair [\[Akbik et al., 2018\]](#):

- trained a character LM
- **use representations from LM like word embeddings**
- train task-specific LSTM on top
- for sequence labeling (PoS tags, entity detection, etc.)

ELMo [\[Peters et al., 2018\]](#)

- word-level LM



# Context-Dependent Word Embeddings



| word       | context                                                   | selected nearest neighbors                                                                                                                                                                   |
|------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Washington | (a) <i>Washington</i> to curb support for [...]           | (1) <b>Washington</b> would also take [...] action [...]<br>(2) <b>Russia</b> to clamp down on barter deals [...]<br>(3) <b>Brazil</b> to use hovercrafts for [...]                          |
| Washington | (b) [...] Anthony <b>Washington</b> (U.S.) [...]          | (1) [...] Carla <b>Sacramento</b> ( Portugal ) [...]<br>(2) [...] Charles <b>Austin</b> ( U.S. ) [...]<br>(3) [...] Steve <b>Backley</b> ( Britain ) [...]                                   |
| Washington | (c) [...] flown to <b>Washington</b> for [...]            | (1) [...] while visiting <b>Washington</b> to [...]<br>(2) [...] journey to New York City and <b>Washington</b> [...]<br>(14) [...] lives in <b>Chicago</b> [...]                            |
| Washington | (d) [...] when <b>Washington</b> came charging back [...] | (1) [...] point for victory when <b>Washington</b> found [...]<br>(4) [...] before <b>England</b> struck back with [...]<br>(6) [...] before <b>Ethiopia</b> won the spot kick decider [...] |
| Washington | (e) [...] said <b>Washington</b> [...]                    | (1) [...] subdue the never-say-die <b>Washington</b> [...]<br>(4) [...] a private school in <b>Washington</b> [...]<br>(9) [...] said <b>Florida</b> manager John Boles [...]                |

Table 4: Examples of the word “Washington” in different contexts in the CONLL03 data set, and nearest neighbors using cosine distance over our proposed embeddings. Since our approach produces different embeddings based on context, we retrieve different nearest neighbors for each mention of the same word.

# LSTM Language Models (1)

Flair and ELMo achieved new state-of-the-art scores across many NLP tasks

But:

- Uses hidden states of LMs only like word embeddings
- Still requires a new LSTM to be trained on top (from scratch)
- Scalability issues of training LSTM language models

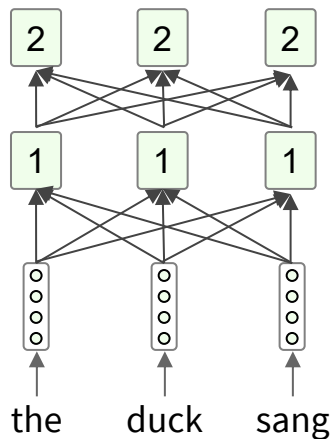
**Solution: Transformer language models?**

# Transformer: High-Level View

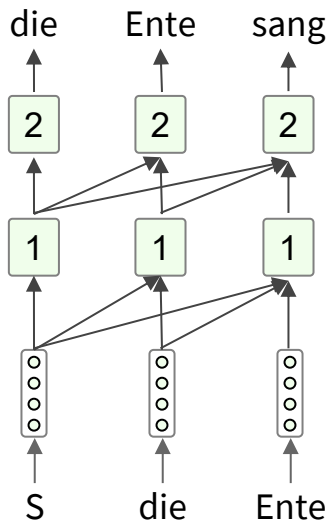
[Vaswani et al., \(2017\)](#) proposed the **transformer** for machine translation

Basic idea: only use attention in encoder + decoder, but no RNN

## transformer encoder



## transformer decoder



*encoder*: many self-attention layers, no masking

*decoder*: many self-attention layers with masking

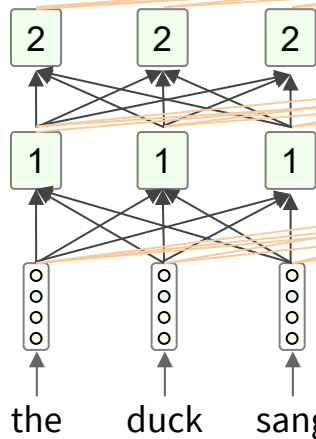
# Transformer: High-Level View



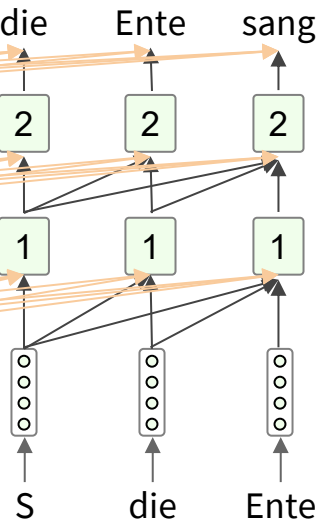
[Vaswani et al., \(2017\)](#) proposed the **transformer** for machine translation

Basic idea: only use attention in encoder + decoder, but no RNN

## transformer encoder



## transformer decoder



*encoder*: many self-attention layers, no masking

*decoder*: many self-attention layers with masking

decoder uses **cross-attention** to encoder!

*Inputs*: Word embeddings + positional encodings

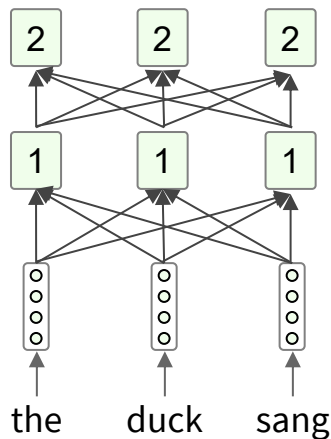


# Language Modeling with Transformers

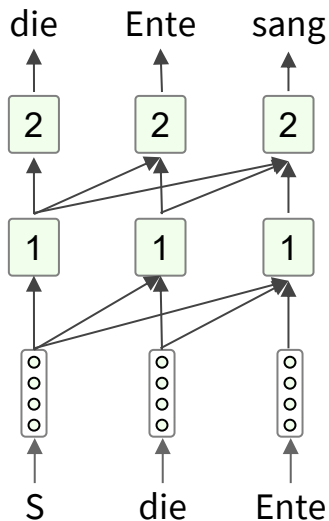


How to use transformer for language modeling?

**transformer encoder**



**transformer decoder**



Approaches:

- use only the decoder (GPT)
- use only the encoder (BERT)

# Outline

New Task: Natural Language Inference

Transfer Learning

- **Causal Language Modeling (GPT)**
- **Masked Language Modeling (BERT)**

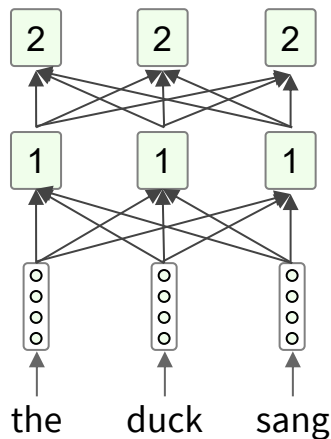
Fine-Tuning vs Linear Probing



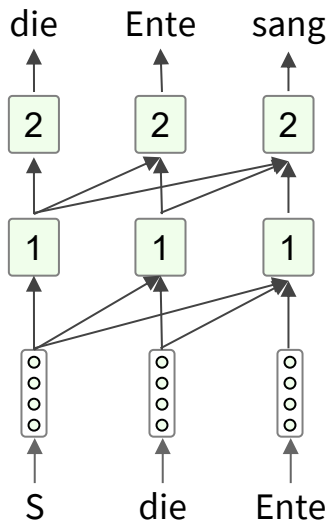
# Language Modeling with Transformers

## How to use transformer for language modeling?

**transformer encoder**



**transformer decoder**



Approaches:

- use only the decoder (GPT)
- use only the encoder (BERT)



# GPT-2: Larger Model trained on More Data



GPT-2 [\[Radford et al., 2019\]](#), a larger version of GPT (1.5B params) trained on more data, was shown to produce relatively convincing samples of natural language

**Context (human-written):** In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

**GPT-2:** The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

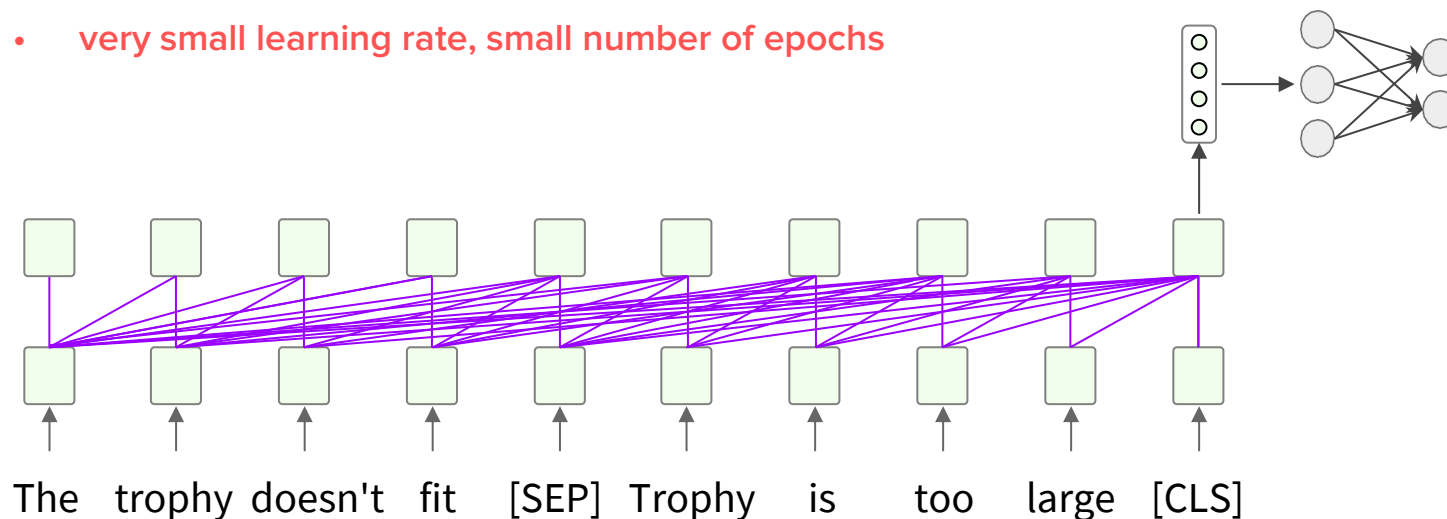
Pérez and the others then ventured further into the valley. "By the time we reached the top of one peak, the water looked blue, with some crystals on top," said Pérez.

# Fine-Tuning for NLI

## Transfer Learning for Natural Language Inference

Take pre-trained model, randomly initialize a softmax classifier on top, and **train both**

- gradient propagates through whole network
- **very small learning rate, small number of epochs**



# GPT for Natural Language Inference



GPT [Radford et al., 2018] results on various natural language inference datasets.

| Method                              | MNLI-m      | MNLI-mm     | SNLI        | SciTail     | QNLI        | RTE         |
|-------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|
| ESIM + ELMo [44] (5x)               | -           | -           | <u>89.3</u> | -           | -           | -           |
| CAFE [58] (5x)                      | 80.2        | 79.0        | <u>89.3</u> | -           | -           | -           |
| Stochastic Answer Network [35] (3x) | <u>80.6</u> | <u>80.1</u> | -           | -           | -           | -           |
| CAFE [58]                           | 78.7        | 77.9        | 88.5        | <u>83.3</u> |             |             |
| GenSen [64]                         | 71.4        | 71.3        | 82.3        | -           | <u>82.3</u> | 59.2        |
| Multi-task BiLSTM + Attn [64]       | 72.2        | 72.1        | -           | -           | 82.1        | <b>61.7</b> |
| Finetuned Transformer LM (ours)     | <b>82.1</b> | <b>81.4</b> | <b>89.9</b> | <b>88.3</b> | <b>88.1</b> | 56.0        |

Table 3: Results on question answering and commonsense reasoning, comparing our model with current state-of-the-art methods.. 9x means an ensemble of 9 models.

Approach beat all other approaches at the time, including ensemble models (5x / 3x)

# Outline

New Task: Natural Language Inference

Transfer Learning

- Causal Language Modeling (GPT)
- **Masked Language Modeling (BERT)**

Fine-Tuning vs Linear Probing

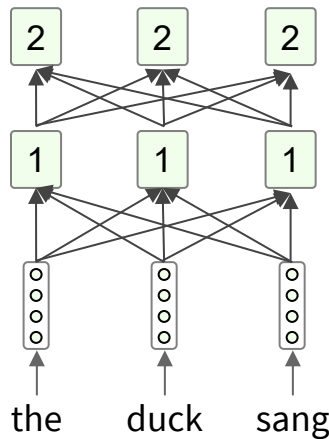




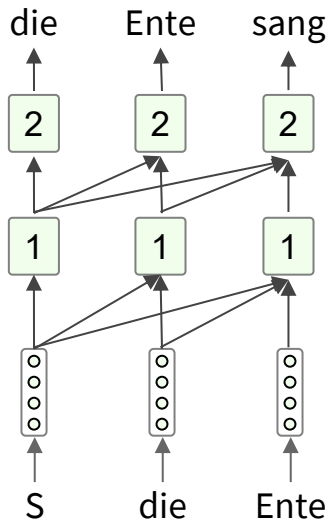
# Language Modeling with Transformers

## How to use transformer for language modeling?

### transformer encoder



### transformer decoder



### Approaches:

- use only the decoder (GPT)
- use only the encoder (BERT)

# Language Modeling with Encoder



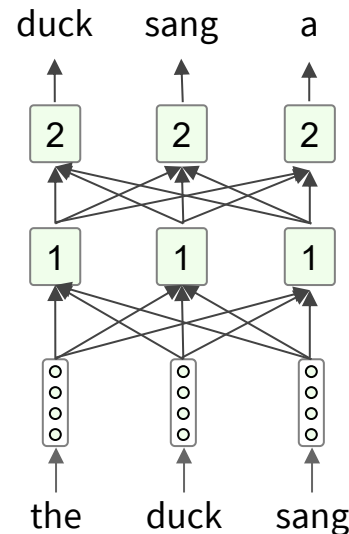
Transformer decoder uses masking

- this limits attention to past words -> no deep bidirectionality!

For many tasks, deep bidirectionality is helpful

- PoS tagging
- sentence-level tasks like encoding in translation

How to do language modeling with a transformer encoder that sees entire sentence at the same time?



# Masked Language Modeling



[Devlin et al., \(2018\)](#) introduced the task of **masked language modeling (MLM)**

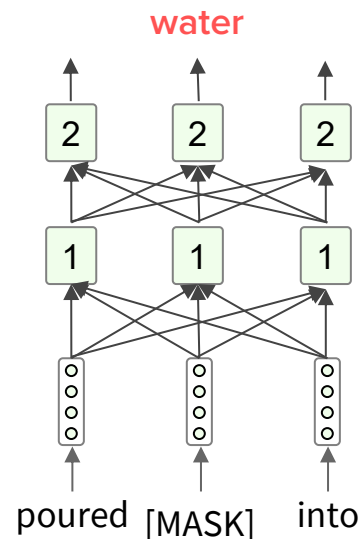
Take text and randomly mask out words

- use special [MASK] token
- model should predict missing words

Example text: I poured water into the cup

- I poured [MASK] into the [MASK]

Essentially: Missing word prediction



# Demo on Huggingface Hub



## ⚡ Inference API ⓘ

📄 Fill-Mask

Examples ▾

Mask token: <mask>

The <mask> was very funny.

Compute

Computation time on cpu: 0.083 s

|         |       |
|---------|-------|
| movie   | 0.126 |
| show    | 0.054 |
| film    | 0.049 |
| episode | 0.041 |
| book    | 0.019 |

# BERT: Pre-trained Transformer Encoder



[Devlin et al., \(2018\)](#) introduced the task of **masked language modeling (MLM)**

Details:

- Predict a random 15% of (sub)word tokens.
  - Replace input word with [MASK] 80% of the time
  - Replace input word with a random token 10% of the time
  - Leave input word unchanged 10% of the time (but still predict it!)
- Why? Doesn't let the model get complacent and not build strong representations of non-masked words. **(No masks are seen at fine-tuning time!)**

# BERT: Pre-trained Transformer Encoder (2)



Additional pre-training objective: **Next sentence prediction (NSP)**

- always input two sentences at the same time, with separator token [SEP]
- predict whether second sentence follows the first
- binary classification task (True/False)

Examples:

- the duck sang [SEP] it sounded beautiful → TRUE
- the duck sang [SEP] I poured water → FALSE

The additional objective was chosen to encourage the model to learn better sentence-level representations

# BERT: Pre-trained Transformer Encoder (3)

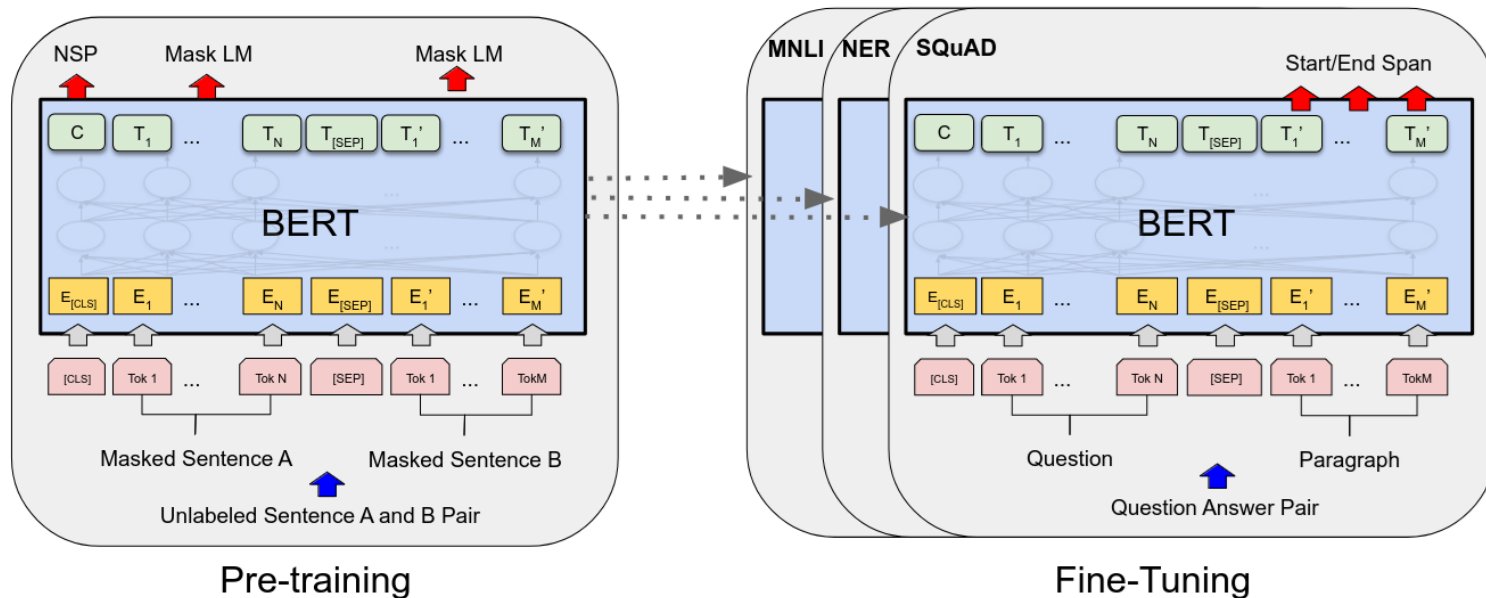


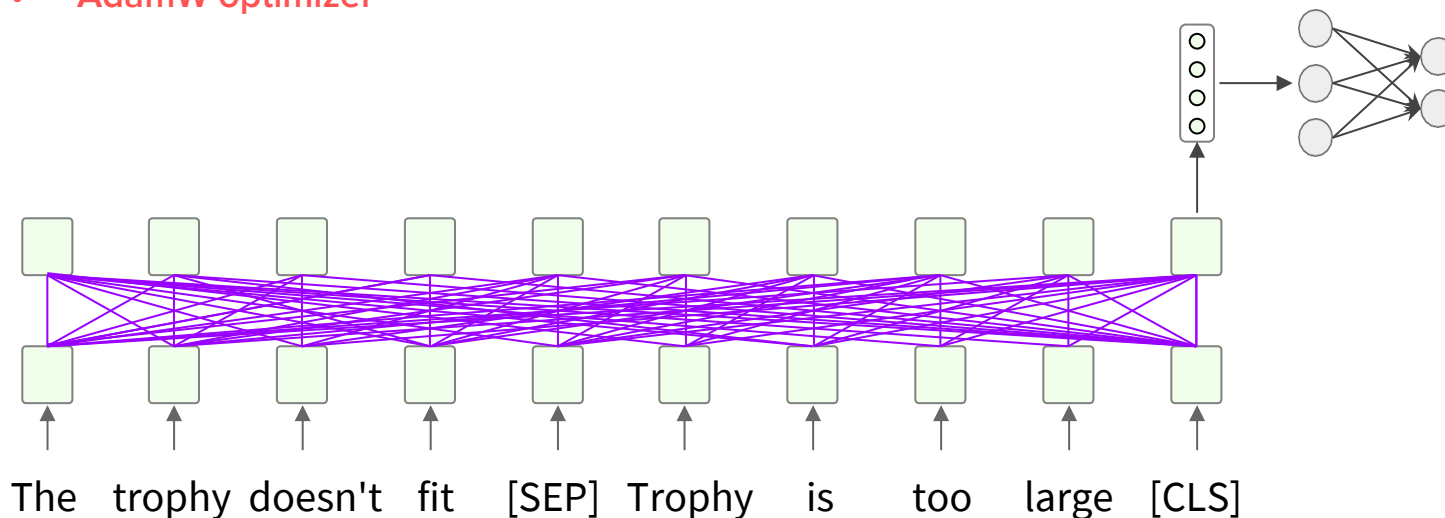
Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned.  $[CLS]$  is a special symbol added in front of every input example, and  $[SEP]$  is a special separator token (e.g. separating ques-

# Fine-Tuning BERT



Take pre-trained BERT, randomly initialize a softmax classifier on top, and **train both**

- gradient propagates through whole network
- **very small learning rate, small number of epochs**
- **AdamW optimizer**





# Fine-Tuning BERT (2)



GLUE benchmark:

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average     |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|-------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0        |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0        |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1        |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6        |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b> |

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.<sup>8</sup> BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

# BERT: Model Sizes and Corpora Used

[Devlin et al., \(2018\)](#) released two models:

- BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params
- BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params

Trained on:

- BooksCorpus (800 million words)
- English Wikipedia (2,500 million words)

**Publicly released models and code to research community!**

Great, because pre-training very expensive, but fine-tuning isn't

- **This enabled the research community to tune pre-trained BERT for all sorts of tasks**

# Transformers Library



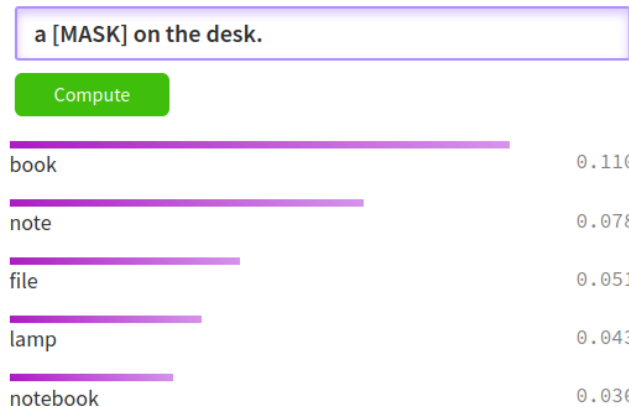
Huggingface's transformers library implements most popular transformer architectures

- [model hub](#) with large pre-trained models (BERT, GPT-2, T5, etc.)

Try the models online!

You don't need to pre-train your own transformer language model:

- just select an appropriate model that someone else trained
- fine-tune it for your task



We learn how with this week's exercise!

# Outline

New Task: Natural Language Inference

Transfer Learning

- Causal Language Modeling (GPT)
- **Masked Language Modeling (BERT)**

Fine-Tuning vs Linear Probing



# Training vs. Fine-Tuning

## Training

- What we have been doing so far
- Model weights mostly initialized randomly (+ maybe word embeddings)
- SGD + large learning rate + many epochs

## Fine-tuning

- Start from a pre-trained transformer language model
- Model weights mostly already initialized (only new classification heads are random)
- Adam optimizer + very small learning rate + few epochs

# Outline

